2017 IEEE/ACM 4th International Workshop on Software Engineering Research and Industrial Practice (SER&IP)

Identifying and Documenting False Positive Patterns Generated by Static Code Analysis Tools

Zachary P. Reynolds*, Abhinandan B. Jayanth*, Ugur Koc†, Adam A. Porter†, Rajeev R. Raje*, James H. Hill*

*Indiana University-Purdue University Indianapolis, Indianapolis, IN, USA
{zpreynol, abjayant, rraje, hilljh}@iupui.edu

†University of Maryland, College Park, MD, USA
{ukoc, aporter}@cs.umd.edu

***Abstract*—This paper presents our results from identifying and documenting false positives generated by static code analysis tools. By false positives, we mean a static code analysis tool generates a warning message, but the warning message is not really an error. The goal of our study is to understand the different kinds of false positives generated so we can (1) automatically determine if an error message is truly indeed a true positive, and (2) reduce the number of false positives developers and testers must triage. We have used two open-source tools and one commercial tool in our study. The results of our study have led to 14 core false positive patterns, some of which we have confirmed with static code analysis tool developers.**

I. Introduction

Static code analysis [1] is the process of checking programs for errors without actually executing them. Developers and testers can use static code analysis to locate flaws in source code (*e.g.*, buffer overflows, null pointer dereferences, *etc.*) that (1) are hard to identify manually, and (2) can eventually lead to security vulnerabilities. For example, the MITRE Corporation [2] manages a list of Common Weakness Enumerations (CWEs) [3], which are weaknesses that can lead to vulnerabilities in software systems. Static code analysis tool developers can then use the list of CWEs as a guideline when developing a static code analysis tool.

There are many static code analysis tools available on the market, both open-source, freely available and commercial tools [4]. The target programming languages and software weaknesses vary between tools. For example, CAT.NET [5] specializes in detecting security flaws in .NET programs, while FindBugs [6] covers a wider range of CWEs in Java [7].

Irrespective of their focus, a characteristic shared by many static code analysis tools is generating large numbers of false positives [8]–[11] (*i.e.*, the tool generates a warning message that is incorrect). We believe the number of false positives is large because (1) code analysis is *hard* and (2) it is often better for the tool to state that there is a problem and be wrong (*i.e.*, a false positive), than to not state that there is a problem and be wrong (*i.e.*, a false negative).

In either case, there is opportunity to reduce the number of false positives generated by static code analysis tools. This is important because triaging large numbers of false positives is time-consuming for developers and may reduce confidence levels in static code analysis tools. Instead, developers need to focus on generated warnings that are true warnings and address them properly. In order to reduce the number of false positives, however, we must first understand the different kinds of false positives generated by static code analysis tools—in the same way CWEs characterize different kinds of vulnerabilities in source code.

With this understanding, the contributions of this paper are as follows:

- We show static code analysis tools flag few recurring false positive patterns when applied to a standardized test suite.
- We standardize how to define false positive patterns using a set of descriptive attributes. The attributes include: the false warning message flagged by the tool; a measure of how often the pattern occurs in tested source code; a minimized source code snippet showing the essence of the false positive; and a description of the pattern.
- We create a hierarchical catalog of false positive patterns to better understand its structure and the variation between similar and different false positive patterns.
- We show the practicality of using a standardized test suite, *e.g.*, the Juliet test suite, to identify false positive patterns.

We performed our study in the context of both open-source and commercial static code analysis tools available from both academia and industry. The results of our study produced a catalog that contains 14 different false positive patterns, some of which we validated with tool developers.

II. Background on False Positives Generated by Static Code Analysis Tools

In Section I we introduced how static code analysis tools are known to generate large numbers of false positives. Likewise, we define a *false positive* as a static code analysis tool generating a warning message for a location in the source code, but the location in question does not have any known problems. To better understand this problem, we present an example false positive message in Listing 1.

978-1-5386-2797-6/17 $31.00 © 2017 IEEE
DOI 10.1109/SER-IP.2017..20

```
1  #include <stdio.h>
2  #define BUFFER_SIZE 10
3
4  int main(void) {
5    char buffer[BUFFER_SIZE] = "";
6    // False positive: Buffer is accessed out of bounds.
7    fgets(buffer, BUFFER_SIZE, stdin);
8    return 0;
9  }
```

Listing 1. False positive example from the Juliet test suite.

In Listing 1 a maximum of nine characters in addition to the null character will be read into the buffer on line 7. Because the program writes to the beginning of the array and the array has space for ten characters, all memory accesses will be valid. Unfortunately, one static code analysis tool warns that the buffer is accessed out of bounds.[1] We regard this warning as a false positive.

There are also cases where a static code analysis tool flags an error that cannot occur for the given calling context but may occur for a different context. For example, in Listing 2, one static code analysis tool warns that a buffer overrun occurs on line 4. However, this program will not cause a buffer overrun because an item is pushed onto the `dataList` structure before the `good()` function is invoked. If the `good()` function, however, is executed in a different context (*e.g.*, the `push_back()` call is omitted on line 9), a buffer overrun may occur. We regard an example like Listing 2 to be a false positive under the given context.

```
1  #include <list>
2  void good(std::list<char *> dataList) {
3    // False positive: Buffer Overrun.
4    char * data = dataList.back();
5  }
6  int main(void) {
7    char * str = "test string";
8    std::list<char *> dataList;
9    dataList.push_back(str);
10   good(dataList);
11   return 0;
12 }
```

Listing 2. False positive example from the Juliet test suite under a particular execution context.

III. Approach to Identifying False Positives

In this section, we describe our approach to identifying false positive patterns. Figure 1 provides an overview of our approach. Each step in the figure is elaborated in the following sections.

Step 1. Select the Static Code Analysis Toolset

The first step was to select the set of static code analysis tools to use in our study. We used three static code analysis tools, as listed in Table I. We have removed the tool names, as our aim is to identify false positive patterns, not afford the opportunity to criticize a particular tool. We included open-source tools because they are freely available and provide a base case to compare against other tools. We included a commercial tool because commercial tools are considered to be more reliable than open-source tools [12].

[1]Adding a check on the return value of `fgets()` still causes the static code analysis tool to display a warning message.

TABLE I. Static code analysis tools selected for our study.

Static code analysis tool	Tool type
Tool A	Open source
Tool B	Open source
Tool C	Commercial

Step 2: Select the Code Base for Analysis

The next step in our process was to select the code base for our study. For this study, we selected the C/C++ Juliet test suite version 1.2 [13] from the National Security Agency (NSA) Center for Assured Software. We selected the Juliet test suite because it was designed specifically for evaluating how static code analysis tools perform against known weaknesses in source code that can lead to security vulnerabilities. Juliet test cases are grouped according to 118 different weaknesses as identified by MITRE's Common Weakness Enumeration [14]. Each CWE is numbered arbitrarily and assigned a name that reflects the concern being documented.

One advantage of using Juliet is that its weaknesses are annotated in the source code, allowing us to quickly identify true positives, false positives, and false negatives generated by a static code analysis tool. Listing 3 shows three kinds of annotations:

- **Potential flaw.** A `POTENTIAL FLAW` annotation denotes the location of the error that the test case targets.
- **Fix.** When possible, the Juliet test suite includes a non-flawed version of a test case that performs the same functionality as the flawed version but does not contain the target flaw. In this case, a `FIX` annotation denotes a change to the source code to remove the flaw.
- **Incidental.** An `INCIDENTAL` annotation marks an unavoidable flaw *not* of the target type flaw.

```
1  static void goodG2B1() {
2    int data;
3    data = -1;
4    if(0) {
5      /* INCIDENTAL: CWE 561 Dead Code */
6      printLine("Benign, fixed string");
7    }
8    else {
9      /* FIX: Use a value not equal to zero */
10     data = 7;
11   }
12   if(1) {
13     /* POTENTIAL FLAW: Possibly divide by zero */
14     printIntLine(100 / data);
15   }
16 }
```

Listing 3. Source code annotations in a Juliet test case.

The main disadvantage of using Juliet is that, since it is a synthetic test suite, members of the community may view it as fabricated test cases that are not representative of production source code [15]. This perception makes it challenging for us to convey our findings to tool vendors (see Section IV-D).

Step 3: Identify the Generated False Positives

As stated earlier, a false positive occurs when a static code analysis tool generates a warning message for a location in the source code, but the location does not have any known errors. Unfortunately, there is no easy (or automated) method for

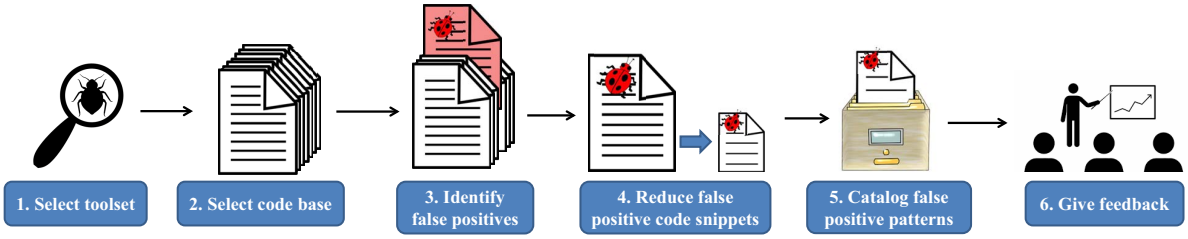

Figure 1. Our general approach for identifying false positives generated by static code analysis tools.

identifying in general whether a generated warning message is indeed a false positive. Fortunately, the Juliet test suite makes it possible to automatically identify a false positive. For example, a warning message is a false positive when there are no annotated flaws (either potential flaws or incidental flaws) on the same line flagged by the warning message.

We therefore leveraged our prior research on evaluating static code analysis tools to quickly locate false positives generated by each static code analysis tool used in our study. More specifically, we integrated each static code analysis tool into SCATE[2], a framework for evaluating the quality of static code analysis tools [16]. Next, we executed the static code analysis tools against the Juliet test suite. We then filtered out the test cases that SCATE identified as producing false positives.

After obtaining the list of test cases generating false positives, we verified that the identified false positives were indeed false positives. For our verification process, we inspected the source code and executed the test cases with dynamic profiling tools, such as Valgrind [17], which can indicate the presence of runtime errors. If our careful inspection and dynamic analysis agreed that the warning was a false positive, then we concluded the warning to be a false positive for the given context. We then studied the program's input to determine the context of the false positive. For example, a warning may be a false positive for a given execution (as in Listing 2) but a true positive in other contexts.

To illustrate this process, Listing 4 shows part of a Juliet test case with a false positive. The value `"Good"` is assigned to the memory location pointed to by the `data` pointer on line 10. On lines 12-15 the value is read through a reference-to-pointer and printed. One static code analysis tool reports a warning on line 9 that a stored value is never read. Executing the program shows that the value `"Good"` is actually read and printed.[3] This warning is therefore a false positive.

Step 4: Reduce the False Positive Source Code

After finding a false positive message, we reduced the source code that generates the false positive. By *reduced*, we mean we removed all elements of the source code not related to the false positive until the removal of the next element resulted in the false positive message going away. We

[2]SCATE is an acronym for Static Code Analysis Tool Evaluator.

[3]Note that printing the value as character data instead of as hex data does not change the static code analysis tool warning.

```
1  #include "std_testcase.h"
2  #include <wchar.h>
3  ...
4  #ifndef OMITGOOD
5  static void goodG2B() {
6    char * data;
7    char * &dataRef = data;
8    /* FIX: Initialize data */
9    // False positive: Value stored to 'data' never read
10   data = "Good";
11   {
12     char * data = dataRef;
13     /* POTENTIAL FLAW: Attempt to use data,
14        which may be NULL */
15     printHexCharLine(data[0]);
16   }
17 }
18 ...
```

Listing 4. Example false positive warning in a Juliet test case.

reduced the source code because it allowed us to better identify the cause of the false positive since only the essence of the problem remained. We call the reduced source code a *false positive pattern*. For example, Listing 5 shows the reduced code of Listing 4. Here are observations about our reduction:

- The false positive is still present in the reduced code. This is a critical point and holds for all reduced false positives.
- Unnecessary functions are removed, and relevant functions are renamed to meaningful names.
- Reduction within relevant functions is performed. For example, the code on lines 12-15 of Listing 4 is moved out of the nested brackets, and the data is read directly from the `dataRef` reference.
- Dependence on Juliet test case support files is eliminated. The header `std_testcase.h` (line 1 of Listing 4) defines reusable types and functions on which test cases depend. We removed this dependency and substituted these types and functions for standard C/C++ ones. For example, the `printHexCharLine()` call on line 15 is replaced with a `printf()` call. The reduced code is more self-contained and allows us to focus on the code structures relevant to the false positive.

Although we have employed manual reduction thus far, we plan to integrate ongoing work on automated code reduction [18] using techniques such as delta debugging. Automated reduction will allow our approach to scale to code sizes larger than Juliet test cases.

Step 5: Catalog the Patterns

After reducing the source code that produces a false positive message by a static code analysis tool, we cataloged the

```
1  #include <stdio.h>
2
3  int main(void) {
4    char * data;
5    char * &dataRef = data;
6    // False positive: Value stored to 'data' never read
7    data = "Good";
8    printf("%c", dataRef[0]);
9    return 0;
10 }
```

Listing 5. Reduced version of Listing 4.

reduced code. The goal of cataloging the false positive pattern was to determine recurring source code structures in the code base that cause static code analysis tools to generate false positives. This approach is similar to documenting software design patterns [19] or cataloging software weaknesses [14].

Each documented false positive pattern has the following attributes:

- The **false positive warning** is the warning message from the static code analysis tool, or tools. We say *tools* because some false positive patterns are prevalent across multiple tools.
- The **description** is a high-level explanation of the code structure causing the false positive.
- The **tools** attribute contains the list of static code analysis tool(s) which reported the false positive pattern.
- **Frequency** is a measure of how often the false positive appears in the code base. There are at least two dimensions in the Juliet test suite: *CWEs* (described in Step 2) and *flow variants*. Flow variants express various control or data flows. To record frequency, we recorded both CWEs and flow variants where the false positive was present.
- The **code sample** is the reduced C/C++ snippet of the false positive. Each code sample is commented with the name of the test case in Juliet where it is derived from.[4] We annotated the false warning on the line immediately before the line which the tool flagged.
- **Changes** are minor modifications to the minimized code samples, along with the static code analysis tool result. We made these changes so that the tool would no longer flag the warning. By comparing the original code with the changes, we can better understand the code structures causing the false positive.

For example, we cataloged the *Buffer Underflow Usage* false positive pattern in Listing 1 as follows:

- **False positive warning**: Buffer is accessed out of bounds.
- **Description**: A value is stored to a buffer after the buffer is initialized. Let L be the length of the value that is stored. The buffer's initial value is less than L-1 in length.
- **Tools**: Tool A[5]
- **Frequency**: Occurs in the following CWEs: 127, 134, 190, 194, 195, 197, 226, 367, 369, 400, 517, 617. Not specific to any particular flow variant.
- **Code sample**: Listing 1 gives the minimized code snippet for this pattern.
- **Changes**: Increasing the length of the buffer's initial value to L-1 causes the false positive to vanish.

We stored the false positive pattern catalog in a GitHub repository. If a newly reduced false positive structure matched an existing pattern in the catalog, then we did not create a new pattern. Instead, we made appropriate changes to the existing pattern's frequency attribute. If a reduced structure was *similar* to an existing pattern but had no match, then we created a generalized description of the false positive with the concrete patterns as variants of the general pattern. This approach produced a hierarchy of false positive patterns, as illustrated in Section IV-A below.

Step 6: Give Feedback to Tool Developers

The last step in our approach was to validate identified false positive patterns by submitting our findings to static code analysis tool developers. We then requested the developers to review submitted artifacts and confirm that each false positive was indeed a false positive. Section IV-D discusses our preliminary results from tool developers.

IV. Results on Identifying and Documenting False Positive Patterns

This section discusses our results on identifying and documenting false positive patterns generated by static code analysis tools.

A. False Positive Hierarchy

Figure 2 shows a hierarchical view of the false positive patterns that we have identified and documented. The patterns are organized as nodes and are given short, descriptive names. We observed that some patterns are prevalent across multiple tools, while others are specific to a single tool. As captured in Figure 2, some patterns are variations of one another—hence the multiple layers. At the first level, there are 14 *distinct* false positive patterns that have no common code structures. The lower level false positive patterns are variations of these core patterns—producing a total of 27 concrete false positive patterns.

For example, the general *Buffer Store* pattern fills a buffer with data from a source. We observed three different data sources for this pattern in Juliet: an environment variable (using `strncat()`), a socket, and the user (using `fgets()`). Therefore, we created three variations of *Buffer Store* corresponding to these data sources. We found these code structures to exhibit a degree of similarity so as to be characterized as the same general pattern, but they had important differences so as to require separate leaf patterns.

B. False Positive Frequencies

Not only did we characterize false positives according to patterns, but we also measured frequencies of these patterns. Table II gives frequencies for the false positive patterns for the three static code analysis tools we tested. The false positive

[4]Comments specifying the derived test cases are removed in the paper to conserve space.

[5]The name of the tool is removed to achieve anonymity.

Figure 2. Hierarchy of identified false positive patterns over the Juliet test suite. The 14 core false positive patterns are numbered in the left-most column.

patterns are listed in the first row. We have numbered the patterns to conserve space, allowing the reader to refer to the pattern names in Figure 2. The table shows the number of times a particular false positive pattern appears in the Juliet test suite. A blank cell indicates that the particular tool does not flag the pattern in Juliet. An `x` indicates that the pattern appears in Juliet, but we have not yet recorded its frequency.[6]

Due to the sheer number of some false positive patterns, we did not verify every warning instance. Instead, we randomly sampled the warning instances to obtain a 95% confidence interval for the lower bound of the number of false positives for each pattern. For example, we are 95% confident that Tool C flags *at least* 428 instances of FP 4 (*Buffer Store*) in the Juliet test suite.

Patterns with higher frequencies carry more significance. Some very high frequencies, such as 3322 for the *Predictable Conditional* pattern, are explained in that these patterns are composed of many variations, according to Figure 2.

[6]Some patterns occur quite frequently, making an accurate frequency measurement difficult to obtain. We are still in the process of gathering data to replace the `x` symbols.

C. False Positive Pattern Examples

In this section, we describe and give code examples of a couple false positive patterns from the catalog.

1) Conditional Mem Leak: As we tested different static code analysis tools, it became clear that global variables were a significant source of false positive warnings. Six of the patterns we cataloged were associated with global variables—specifically global variables used in conditions.

In Listing 6, we illustrate this concept via an example of the *Conditional Mem Leak* pattern. This snippet shows the reduced code (the original test case in the Juliet test suite had more code surrounding this structure). First, a `char` array is allocated memory. Next, the condition tests the value of a global variable, and the memory is freed if the global variable evaluates to `true`. Because the global variable is initialized to `1` and never changed, static code analysis tools should be able to analyze that the memory will always be freed and no memory leak will occur. Two static code analysis tools, however, flagged a memory leak false positive for this example.

```
1  #include <stdlib.h>
2  static int staticTrue = 1;
3
4  int main(void) {
5    char * data = (char *)malloc(10*sizeof(char));
6    if(staticTrue) {
7      free(data);
8    }
9    // False positive: Memory leak
10 }
```

Listing 6. *Conditional Mem Leak* false positive pattern.

2) File Close Virtual Method: Some false positive patterns involve classes (see Listings 7-9). Listing 7 defines a `FileCloserBase` interface. The class `FileCloserSub` implements the `FileCloserBase` interface to close a file given the file descriptor (see Listing 8). The client code in Listing 9 opens a file, creates an instance of `FileCloserSub`, and passes the file descriptor to the method where it is safely closed. One static code analysis tool flags a false positive "Leak" warning, claiming the file descriptor is never closed.

```
1  // FileCloser.h
2  #include <unistd.h>
3  #include <stdio.h>
4
5  namespace files {
6
7  class FileCloserBase {
8  public:
9    virtual void action(int fildes) {};
10 };
11
12 class FileCloserSub : public FileCloserBase {
13 public:
14   void action(int fildes);
15 };
16
17 }
```

Listing 7. *File Close Virtual Method* false positive pattern header file.

Besides `open()`, we replicated this false positive across two additional C allocators: `open64()` and `creat()`.

TABLE II. Frequency of false positives in sampled test cases from the Juliet test suite. The 14 core false positive patterns are numbered in the first row and refer to the pattern names in Figure 2.

	FP 1	FP 2	FP 3	FP 4	FP 5	FP 6	FP 7	FP 8	FP 9	FP 10	FP 11	FP 12	FP 13	FP 14
Tool A					x					68	3322	1456		
Tool B									x	x	x			x
Tool C	240	19	12	428		61	2	63					2	

```
1  // FileCloser.cpp
2  #include "FileCloser.h"
3  #include <unistd.h>
4  namespace files {
5  void FileCloserSub::action(int fildes) {
6    close(fildes);
7  }
8  }
```

Listing 8. *File Close Virtual Method* false positive pattern implementation file.

```
1  // client.cpp
2  #include "FileCloser.h"
3  #include <fcntl.h>
4  #include <stdio.h>
5  #include <sys/stat.h>
6  namespace files {
7  void createFile(void) {
8    int fildes = open("file.txt", O_RDWR|O_CREAT, S_IREAD|
       ↪ S_IWRITE);
9    FileCloserBase * fileCloser = new FileCloserSub();
10   fileCloser->action(fildes);
11   delete fileCloser;
12 }
13 // False positive: Leak. `fildes` has gone out of scope
14 // and no longer references the resource of interest.
15 }
16 int main(void) {
17   files::createFile();
18   return 0;
19 }
```

Listing 9. *File Close Virtual Method* false positive pattern client file.

D. Feedback from Static Code Analysis Tool Developers

In Step 6 of Section III, we mentioned that we validated false positive patterns in our catalog by requesting static code analysis tool developers to confirm each false positive to indeed be a false positive. We initially submitted the pattern *File Close Virtual Method* (see Section IV-C2) to the vendor of Tool C, a commercial tool. The developer was initially inclined to dismiss the false positive pattern because the Juliet test suite is not real-world code and is therefore sometimes regarded as contrived. The developer, however, confirmed this pattern to be a false positive and is in the process of creating a patch for the static code analysis tool.

The developer then requested the remaining false positive patterns from our catalog. We submitted 8 total patterns. At the time of this writing, here are the responses we received from the developer for the false positive patterns we submitted:

- 3 patterns were confirmed to be false positives. These patterns are *File Close Virtual Method*, *Buffer Store Socket*, and *Sscanf Uninit Var*.
- 1 pattern was characterized as a true positive. Although the error cannot arise in the context of the Juliet test suite, the error may be a true positive in other contexts. This pattern is *Alloc Using External Size*.
- 1 pattern could not be reproduced. The developer believed the reason was a difference in C/C++ library versions between our team and the developer's system. This pattern is *List Overrun*.
- We have not heard back from the developer regarding 3 patterns, which are *Array Input Conditional*, *Array Resize*, and *Clear List*.

V. Related Work

This section compares our work to existing work on false positive documentation.

Ayewah et al. [11] investigated different kinds of warnings generated by FindBugs in the context of source code from Google's Java Code base, Sun's JDK [20], and Sun's Glassfish [21]. As part of their study, they classified warning messages as false positives, trivial bugs, and serious bugs. Our work extends their work in that we focus on cataloging false positives generated by static code analysis tools.

Yuksel and Sozer [22] evaluated an approach that uses machine learning techniques to classify warnings reported by static code analysis tools based on a set of 10 artifact characteristics: severity, alert code, lifetime, developer idea, file name, module name, open alerts, total alerts, total alerts in module, and total alerts in file. They used the Digital TV software system maintained by Vestel Electronics [23] as the test data set, and they classified alerts as true positives or false positives. Our effort extends their effort in that we further classify false positive warnings generated by static code analysis tools.

de Mendonca et al. [24] conducted a mapping study of current static code analysis tools, which included studying 64 research papers on reducing the number of false positives in tools. The authors classified the studies based on the type of false positive addressed in the source paper using a scheme designed by Basili and Selby [25]. Our work is similar to work done by de Mendonca et al. because we focus on cataloging false positives according to patterns, as shown by the false positive catalog we have constructed. However, our work differs in that we identify and catalog false positives by studying current warnings generated by static code analysis tools, whereas de Mendonca et al. classified false positives by reviewing literature about known false positives. More importantly, our work extends their effort because we detail how to transform source code that generates false positives to its essence (Step 4 in Section III discusses our code reduction approach).

VI. Threats to Validity

Here are some current limitations of our work and how we plan to address them.

- **Limited data.** As mentioned in Section IV-B, we do not have complete data for the frequencies of false positive patterns in our catalog. We are, however, gathering more data for our current toolset. Also, we plan to expand our toolset with additional static code analysis tools to reduce bias of any particular tool.
- **Code base selection.** We realize the Juliet test suite may not be representative of real code, so we plan to test our approach on open-source libraries, such as Coreutils [26] or POCO [27]. Whether the same false positive patterns (or new patterns) emerge remains to be seen.
- **Lack of a pattern specification.** In order to efficiently check larger programs for the presence of false positive patterns, we need to specify a language-independent metamodel for describing patterns. We have prototyped a pattern recognizer to identify the *Conditional Mem Leak* pattern, a relatively simple pattern. As a preliminary experiment, we randomly sampled 100 "memory leak" warnings flagged by Tool A in one CWE of Juliet. Of these warnings, 35 were instances of the *Conditional Mem Leak* pattern. Our recognizer correctly flagged all 35 instances and 27 others in this CWE, all of which we manually verified to contain the false positive pattern. More research is needed to learn whether we can build accurate recognizers for more complex patterns as well.

VII. Concluding Remarks

In this paper, we presented an approach to identifying false positive patterns in static code analysis. Our catalog of 14 false positive patterns shows that a large number of warnings in the Juliet test suite could be characterized into a small number of patterns. Many such patterns occur frequently, and the numbers may grow as we integrate new tools and code bases into our study. We tested both open-source and commercial static code analysis tools and observed some patterns to be common across tools. We showed that it is possible to infer program structures that tend to cause false positives. We also demonstrated the practicality of the Juliet test suite in helping tool developers discover bugs in their tools. Lastly, we have published our current catalog of identified false positives at the following location: https://github.com/SEDS/mangrove-catalog.

Acknowledgment

This work was sponsored by the Department of Homeland Security under grant #D15PC00169.

References

[1] P. Louridas, "Static code analysis," *IEEE Software*, vol. 23, no. 4, pp. 58–61, 2006.
[2] "The MITRE corporation," https://www.mitre.org/.
[3] "Common Weakness Enumeration frequently asked questions," http://cwe.mitre.org/about/faq.html, accessed: 2016-09-13.
[4] Wikipedia, "List of tools for static code analysis — Wikipedia, the free encyclopedia," 2016, [Online; accessed 13-September-2016]. [Online]. Available: https://en.wikipedia.org/w/index.php?title=List_of_tools_for_static_code_analysis&oldid=739038439
[5] "Microsoft code analysis tool .NET (CAT.NET) v1 CTP - 32 bit," https://www.microsoft.com/en-us/download/details.aspx?id=19968, accessed: 2016-09-18.
[6] "Findbugs - find bugs in Java programs," http://findbugs.sourceforge.net/, accessed: 2016-09-19.
[7] "A look at CWE coverage across open source and commercial static analysis tools," https://codedx.com/a-look-at-cwe-coverage-across-open-source-and-commercial-static-analysis-tools/?v=7516fd43adaa, accessed: 2016-09-13.
[8] M. Nadeem, B. J. Williams, and E. B. Allen, "High false positive detection of security vulnerabilities: A case study," in *Proceedings of the 50th Annual Southeast Regional Conference*. ACM, 2012, pp. 359–360.
[9] N. Antunes and M. Vieira, "Comparing the effectiveness of penetration testing and static code analysis on the detection of SQL injection vulnerabilities in web services," in *Dependable Computing, 2009. PRDC'09. 15th IEEE Pacific Rim International Symposium on*. IEEE, 2009, pp. 301–306.
[10] M. Zitser, R. Lippmann, and T. Leek, "Testing static analysis tools using exploitable buffer overflows from open source code," in *ACM SIGSOFT Software Engineering Notes*, vol. 29, no. 6. ACM, 2004, pp. 97–106.
[11] N. Ayewah, W. Pugh, J. D. Morgenthaler, J. Penix, and Y. Zhou, "Evaluating static analysis defect warnings on production software," in *Proceedings of the 7th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*, ser. PASTE '07. New York, NY, USA: ACM, 2007, pp. 1–8. [Online]. Available: http://doi.acm.org/10.1145/1251535.1251536
[12] N. Ayewah, D. Hovemeyer, J. D. Morgenthaler, J. Penix, and W. Pugh, "Using static analysis to find bugs," *IEEE software*, vol. 25, no. 5, pp. 22–29, 2008.
[13] "Software assurance reference dataset," https://samate.nist.gov/SRD/testsuite.php, accessed: 2016-09-19.
[14] The Common Weakness Enumeration (CWE) Initiative, MITRE Corporation. http://cwe.mitre.org/.
[15] A. Delaitre, B. Stivalet, E. Fong, and V. Okun, "Evaluating bug finders: Test and measurement of static code analyzers," in *Proceedings of the First International Workshop on Complex faUlts and Failures in LargE Software Systems*. IEEE Press, 2015, pp. 14–20.
[16] L. M. R. Velicheti, D. C. Feiock, M. Peiris, R. Raje, and J. H. Hill, "Towards modeling the behavior of static code analysis tools," in *Proceedings of the 9th Annual Cyber and Information Security Research Conference*, ser. CISR '14. New York, NY, USA: ACM, 2014, pp. 17–20. [Online]. Available: http://doi.acm.org/10.1145/2602087.2602101
[17] "Valgrind," http://valgrind.org/, accessed: 2016-09-19.
[18] M. Marenchino, "Source code reduction to summarize false positives," Ph.D. dissertation, 2015.
[19] J. O. Coplien, "Software design patterns," in *Encyclopedia of Computer Science*. Chichester, UK: John Wiley and Sons Ltd., pp. 1604–1606. [Online]. Available: http://dl.acm.org/citation.cfm?id=1074100.1074801
[20] "Java development kit 8," http://www.oracle.com/technetwork/java/javase/downloads/index.html, accessed: 2016-09-19.
[21] "Glassfish open source edition 4.1.1," http://www.oracle.com/technetwork/java/javaee/downloads/index.html, accessed: 2016-09-19.
[22] U. Yuksel and H. Sozer, "Automated classification of static code analysis alerts: A case study," in *Software Maintenance (ICSM), 2013 29th IEEE International Conference on*, Sept 2013, pp. 532–535.
[23] "Vestel electronics," http://vestelinternational.com/en/, accessed: 2016-09-23.
[24] V. R. L. de Mendonca, C. L. Rodrigues, F. A. A. de MN Soares, and A. M. R. Vincenzi, "Static analysis techniques and tools: A systematic mapping study," 2013.
[25] V. R. Basili and R. W. Selby, "Comparing the effectiveness of software testing strategies," *IEEE transactions on software engineering*, no. 12, pp. 1278–1296, 1987.
[26] "Coreutils - GNU core utilities," https://www.gnu.org/software/coreutils/coreutils.html, accessed: 2017-02-25.
[27] "POCO C++ libraries," https://pocoproject.org/, accessed: 2017-02-25.